Design of an Arithmetic-Logical Unit

Șchiopu Denisa-Patricia

Group 30431

Contents

1. **Introduction**
   - 1.1 Arithmetic Logic Unit
   - 1.2 Problem
   - 1.3 Motivation and Objectives
   - 1.4 Project Plan

2. **Bibliographic Study**

3. **Analysis**
   - 3.1 Arithmetic Operations
   - 3.2 Logical Operations

4. **Design**
   - 4.1 Arithmetic Operations
   - 4.2 Logical Operations

5. **Implementation**
   - 5.1 Arithmetic Unit
   - 5.2 Logic Unit
   - 5.3 Shifter
   - 5.4 Multiplier
   - 5.5 Divider
   - 5.6 Control Unit
   - 5.7 Execution Unit
   - 5.8 Memory Unit
   - 5.9 Debouncer
   - 5.10 Seven Segment Display Driver
   - 5.11 Top Level Module

6. **Testing and Validation**

7. **Conclusion**

8. **References**

1. Introduction

1.1 Arithmetic Logic Unit

An **Arithmetic Logic Unit (ALU)** [1] is a combinational digital circuit that performs arithmetic and bitwise operations on integer binary numbers. It is a fundamental building block of many types of computing circuits, including the central processing unit (CPU). It is where the circuitry to perform value comparisons and arithmetic computations on information stored in registers occurs. Typical operations of an ALU are arithmetic such as division, addition and subtraction, logical, comparisons and shift or rotate operations.

The purpose of an ALU is to speed up a CPU's overall processing by performing math and logic functions. In early microprocessors, the main CPU could only perform basic operations, and more complex processes, like math, required many steps and took a long time to perform. Modern CPU cores use a pipelining approach to work on multiple things at the same time. Most processors have more than one ALU, being able to perform multiple operations simultaneously.

1.2 Problem

As technology continues to evolve, the design of efficient, fast, and reliable ALUs has become increasingly important for improving system performance. Inefficient designs can lead to slower computations, higher energy usage, and greater circuit complexity. Students and engineers need to understand how to optimize these factors when designing digital systems.

1.3 Motivation and Objectives

The main objective of this project is to design and implement a functional Arithmetic Logic Unit capable of performing basic arithmetic and logical operations, specifically addition and subtraction in C2, increase, decrease, AND, OR, NOT, negation and rotation, as well as multiplication and division implemented as distinct circuits.

The ALU will be designed using simulation tools such as Vivado to model, test, and verify the design’s correctness and efficiency. Through this approach, the project aims to produce a design that is educational, practical and efficient, demonstrating how optimized logic design can improve computational performance while maintaining low hardware complexity.

1.4 Project Plan

Project Meeting 1: Choosing topic, writing introduction and bibliographic study.

Project Meeting 2: Analysis of existing ALUs.

Project Meeting 3: Design (block diagrams).

Project Meeting 4: Implementation of the ALU.

Project Meeting 5: Testing and validation of the designed architecture.

Project Meeting 6: Modifications and final adjustments if needed.

2. Bibliographic Study

While this overarching function remains the same, arithmetic logic units (ALUs) can be of the following types:

- Integer based: used to execute elementary mathematical operations such as additions, subtractions, multiplications, and divisions.
- Bit-slice structure: composed of many smaller ALUs, each responsible for executing operations on a distinct collection of bits, allowing the ALU to operate on larger values.
- Hybrid: seen mostly in more advanced processors, it is a mixture of many kinds of ALUs and is intended to execute a wide variety of logical and arithmetic functions.

In 1945, the mathematician John von Neumann presented the ALU idea in a study on the EDVAC computer's underpinnings [4].

Due to the high cost, size, and power consumption of electronic circuitry throughout the infancy of the Information Age, most early computers featured a serial ALU that simultaneously worked on a single data bit while presenting programmers with a larger word size. The 1948 Whirlwind I [5] was one of the early computers with several ALU circuits and the ability to process 16-bit words.

Fairchild created the very first ALU constructed as an integrated circuit in 1967 with the Fairchild 3800, which consisted of an 8-bit ALU with only an accumulator. Other integrated ALUs followed soon, these devices being typically "bit slice" capable. Full integrated-circuit ALUs soon emerged, including four-bit ALUs such as Am2901 [6] and

74181 [7]. Microprocessors began to appear in the early 1970s. Later, when transistor geometries shrank following Moore's law, it became feasible to build wider ALUs. Numerous ALUs of the present day have broad word widths and features like barrel shifters or binary multipliers.

In [8] some different implementations of the ALU operations are mentioned, as well as optimization methods depending on the choice of adder architecture, logic style and multiplication and division implementation method. The paper in [10] also compares the performance of the same 32-bit processor using two different adders, an RCA and a Sklansky adder.

Two's complement is the most common method of representing signed integers on computers. two's complement uses the most significant bit as the sign to indicate positive (0) or negative (1) numbers, and nonnegative numbers are given their unsigned representation. However, negative numbers are represented by taking the bit complement of their magnitude and then adding one [13].

3. Analysis

This chapter will describe the arithmetic and logic operations performed by the arithmetic-logical unit, as well as the algorithms used to implement them.

3.1 Arithmetic Operations

3.1.1 Addition

Two’s complement is used because it allows both positive and negative integers to be represented and processed using the same binary addition logic. The equations for a 1-bit full adder are:

$$S = X_1 \oplus X_2 \oplus Cin$$

$$Cout = X_1 * X_2 + (X_1 \oplus X_2)\ Cin$$

Overflow occurs when the value of the inputs is such that there is a carry which changes the most-significant sign bit. When adding, if the sign of the two inputs is the same, but the result sign is different, then we have an overflow. For a 32-bit adder, the overflow is determined using the formula:

$$overflow = Cout_{31} \oplus Cin_{31} = Cout_{31} \oplus Cout_{30}$$

3.1.2 Subtraction

In twos' complement we can determine a negative number by inverting all the bits of its positive representation and adding 1. Therefore, subtraction can be implemented using the addition logic, setting the carry to 1 and negating the subtracted input:

$$\text{A-B} = \text{A} + (\text{not}(\text{B}) + 1)$$

The overflow for the subtraction of 32-bit numbers is determined by the equation:

$$overflow = \text{A}_{31} * \text{not}(\text{B}_{31}) * \text{not}(\text{S}_{31}) + \text{not}(\text{A}_{31}) * \text{B}_{31} * \text{S}_{31}$$

3.1.3 Increment

Increment is the operation which increases the input by 1. It can be implemented using addition:

$$\text{X} = \text{X} + 1$$

3.1.4 Decrement

Decrement is the operation which decreases the input by 1. It can be implemented using subtraction, where the subtracted value is 1:

$$\text{X} = \text{X} - 1$$

3.1.5 Negation

Negation produces the two's complement of the input. This changes the sign of the number, converting positive values to negative and vice versa. The operation is performed by inverting all bits of the accumulator and adding one to the result:

$$-\text{X} = \text{not}(\text{X}) + 1$$

3.1.6 Multiplication

There are multiple algorithms that can be used to implement multiplication in an ALU. A Wallace Tree Multiplier is more complex than the Shift-And-Add Multiplier and requires more power, so I will use the Shift-And-Add Multiplier in this project due to its simpler implementation. The idea is to add the multiplicand to itself

repeatedly. The algorithm is based on taking each digit of the multiplicand in turn and multiplying it by a single digit of the multiplier. Each intermediate product is placed in the appropriate positions, to the left of the earlier results. Finally, all the intermediate products are added together, to get the final result. The steps for this algorithm are illustrated here:

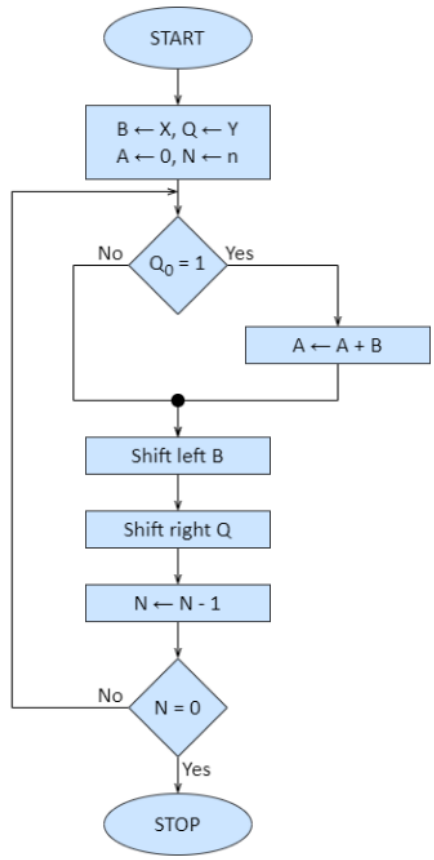

Figure 1. Shift and Add Algorithm

3.1.7 Division

In a division operation, the dividend (X) is divided by the divisor (Y) to produce a quotient (Q) and a remainder (R), following the relationship:

$$X = Q \cdot Y + R \text{ with } R < Y$$

The Restoring Division algorithm: the A register (partial remainder) is initially cleared, while the dividend is loaded into the Q register and the divisor into its register. During each iteration, the combined A–Q register is shifted left by one bit then the divisor is subtracted from A. If the result is negative, the original value of A

is restored and a 0 is placed in the least significant bit of Q. If the result is positive or zero, the subtraction is kept and a 1 is placed in the least significant bit of Q. After all bits have been processed, the Q register contains the quotient, and the A register holds the remainder. This version of the algorithm is faster and simpler, since it avoids shifting the divisor and combines the A and Q registers into a single shifting unit. The steps of the algorithm can be seen in this schematic:

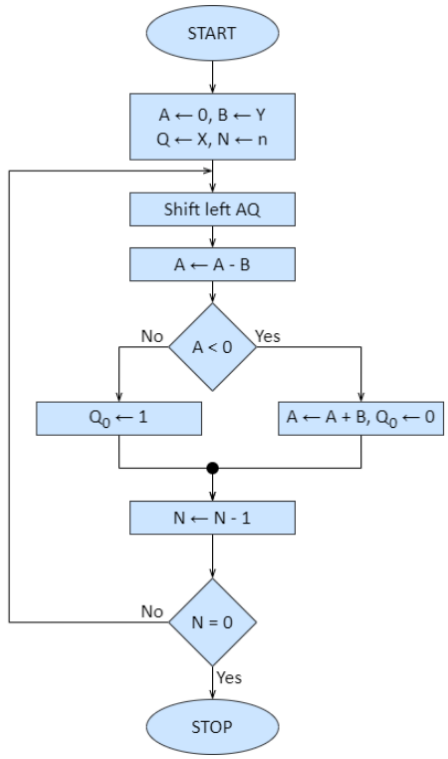

Figure 2. Restoring Division Algorithm

3.2 Logical Operations

3.2.1 AND

This operation performs a bitwise AND operation between the inputs. Each bit in the result is set to 1 only if both corresponding bits of the operands are 1. The truth table of the operation is:

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

Figure 3. Truth table for AND

3.2.2 OR

This operation performs a bitwise OR operation between the inputs. Each bit in the result is set to 1 if at least one of the corresponding operand bits is 1. The truth table of the operation is:

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

Figure 4. Truth table for OR

3.2.3 NOT

This operation performs a bitwise inversion (complement) of the accumulator contents. All 1 bits become 0, and all 0 bits become 1.

A	B
0	1
1	0

Figure 5. Truth table for NOT

3.2.4 Rotate Left (ROL)

This operation rotates the bits of the input to the left by one position. The most significant bit (MSB) is wrapped around to become the least significant bit (LSB).

$$Y = X_{30}X_{29}...X_0X_{31}$$

3.2.5 Rotate Right (ROR)

This operation rotates the bits of the accumulator to the right by one position. The least significant bit (LSB) is wrapped around to become the most significant bit (MSB).

$$Y = X_0X_{31}...X_2X_1$$

4. Design

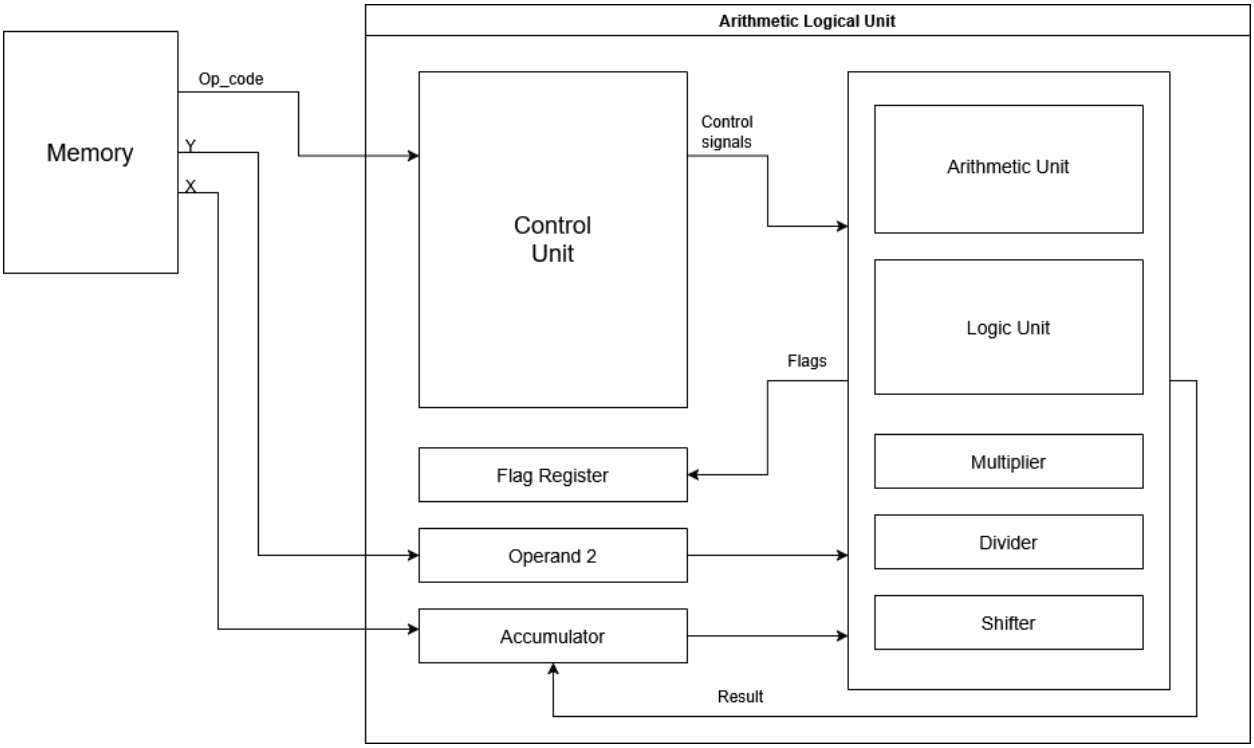

Figure 6. Alu block schematic

The design adopts an accumulator-based architecture, where one operand and the result are stored in a dedicated register. The ALU supports a comprehensive set of operations: addition and subtraction in two's complement form, increment and decrement, logical functions (AND, OR, NOT), negation, and rotation (left and right). Multiplication and division are implemented as distinct external circuits, while a memory unit provides storage and retrieval of operands and intermediate results.

The accumulator is a 32-bit register that serves as the principal operand source and as the destination for the ALU's computed results.

The secondary operand register temporarily stores the second input for binary operations.

The flag register stores the condition flags generated by the ALU after each operation:

- Z – zero flag: Set when the result of an operation is equal to zero.

- C – carry flag: Indicates an unsigned carry out of the most significant bit in addition, or a borrow in subtraction.
- P – parity flag: Set when the result contains an even number of logic-1 bits.
- O – overflow flag: Set when a signed arithmetic overflow occurs.
- A - auxiliary carry flag
- S – sign flag: Indicates that the operation result was saturated instead of overflowing.
- D – divide by zero flag: Set when a division operation is attempted with a divisor equal to zero.

The control unit coordinates all ALU operations. It decodes operation codes, activates the corresponding functional unit, manages data transfers between registers and memory, and generates control signals for the result multiplexer and flag updates.

The arithmetic unit performs addition and subtraction in two's complement form as well as increment, decrement, and negation, thereby maximizing functional reuse of the arithmetic hardware.

The logic unit performs the bitwise operations AND, OR and NOT.

Multiplication and division are realized as dedicated external circuits that implement the operations algorithm.

The Shifter performs the rotate right and rotate left operations.

4.1 Arithmetic Operations

4.1.1 Addition

To perform addition between two 32-bit numbers we need a 32-bit adder, which is built from 32 one-bit adders connected in a serial way. The design is presented in Figure 7.

4.1.2 Subtraction

I will use the same component from the addition. Set the initial carry in to 1 instead of 0, thus adding an extra 1 to the sum and instead of using NOT gates, we will use XOR gates to negate the subtracted operand, as seen in Figure 8.

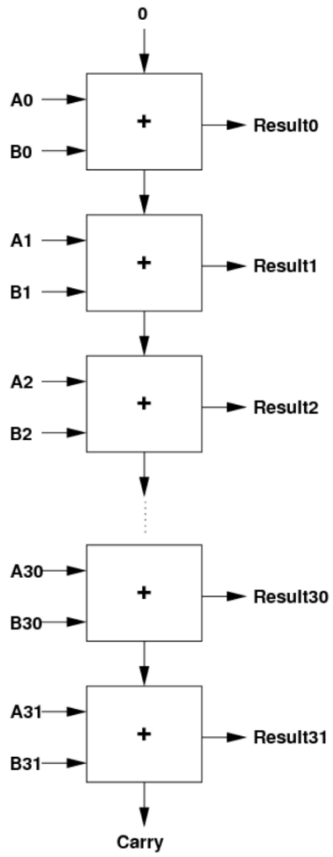

Figure 7. 32-bit adder

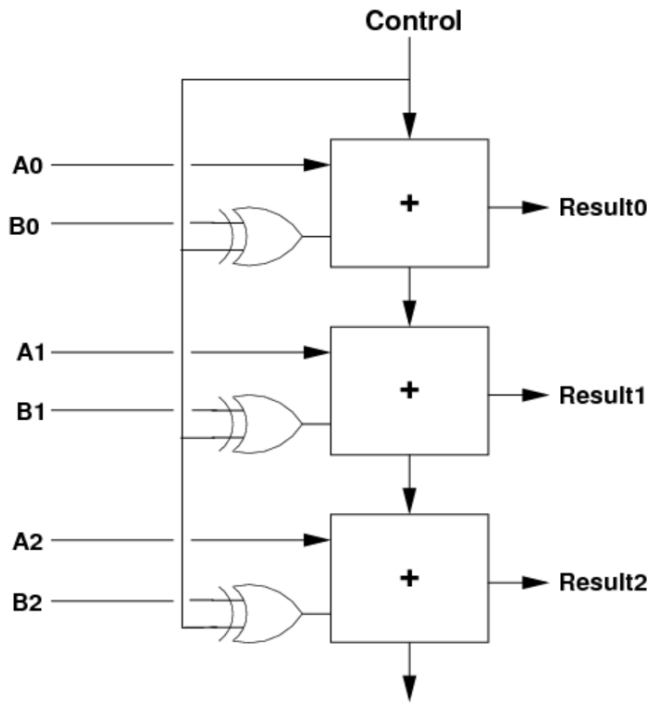

Figure 8. The added logic for subtraction

4.1.3 Increment

This operation is an addition between the value 1 in the accumulator operand and the value stored in the second operand. Since it has the same logic as addition, I will use the same component and set the value in the accumulator to 1.

4.1.4 Decrement

This operation is a subtraction so I will use the same component and logic again and set the subtracted value to 1.

4.1.5 Negation

The negation of an operand is implemented using the same hardware path as subtraction. The first operand is set to 0 and the first carry in is set to 1.

4.1.6 Multiplication

The multiplication unit is implemented as a distinct sequential circuit that utilizes the Shift-and-Add algorithm described in the Analysis chapter. This operation requires multiple clock cycles to compute the final product. The design utilizes three primary registers and a 32-bit adder: B (multiplicand), Q (multiplier), A (accumulator) and a counter N to track the required number of iterations.

At the start of the operation, the control unit loads the operands into registers B and Q. In each iteration, the logic checks the value of Q's LSB. If it is 1, the internal B value is added to A. Then, B is shifted left and Q is shifted right. The counter is decremented, and the process repeats until it reaches 0, at which point Register A holds the final product.

Since the ALU operates on signed integers using the Two's Complement representation, and the Shift-and-Add algorithm functions on magnitudes, the multiplication requires sign handling logic. At the start of the operation, the internal logic checks the MSB of both the Accumulator (Multiplier) and Operand 2 (Multiplicand). The final sign of the result is determined by the XOR of these two sign bits. Before the Shift-and-Add process begins, the operands are converted to their absolute values using negation. After the operation is performed, depending on what sign it needs to have, it is either negated or left as is.

4.1.7 Division

The division unit is implemented as a distinct sequential circuit that executes the Restoring Division algorithm. Similarly to the multiplication, it takes multiple clock cycles and requires additional sign handling logic. The sign of the quotient is determined by the XOR of the operand signs, while the sign of the remainder inherits the sign of the dividend.

Once the operands are converted to positive magnitudes, the Restoring Division algorithm is executed over 32 clock cycles. The dividend is loaded into the lower 32 bits (Register Q) of the combined shift register, while the upper 32 bits (Register A), serving as the partial remainder, are initialized to zero. In each iteration, the combined register is shifted left by one bit, and the divisor is subtracted from the partial remainder. If the subtraction results in a negative value, the logic restores the previous remainder by adding the divisor back and sets the least significant bit of the quotient to zero. Conversely, if the result is positive or zero, the subtraction is preserved, and the quotient bit is set to one.

4.2 Logical Operations

4.2.1 AND

To build the logic for a 32-bit AND I used 32 1-bit AND gates, one for each pair of bits from the operands.

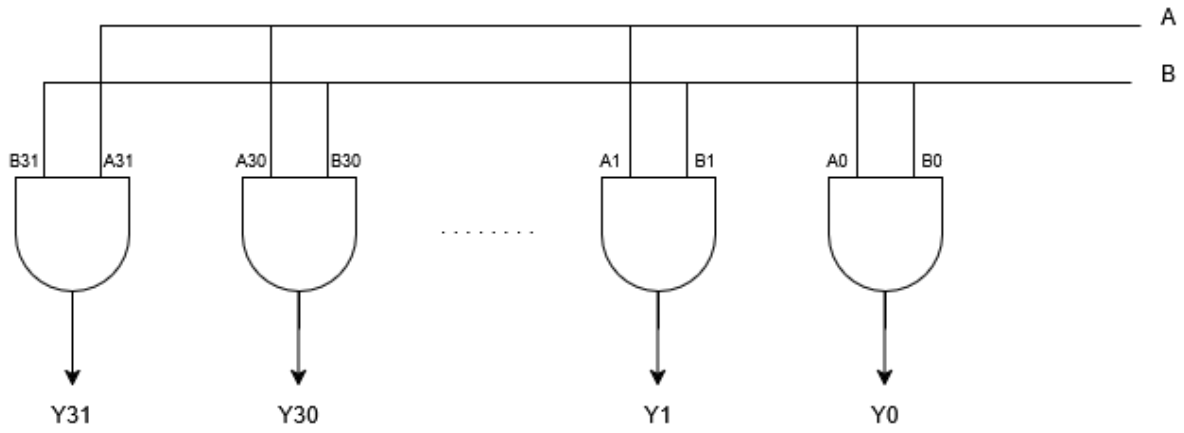

Figure 9. AND for 32 bits

4.2.2 OR

To build the logic for a 32-bit OR I used 32 1-bit OR gates, one for each pair of bits from the operands, the same approach as for AND.

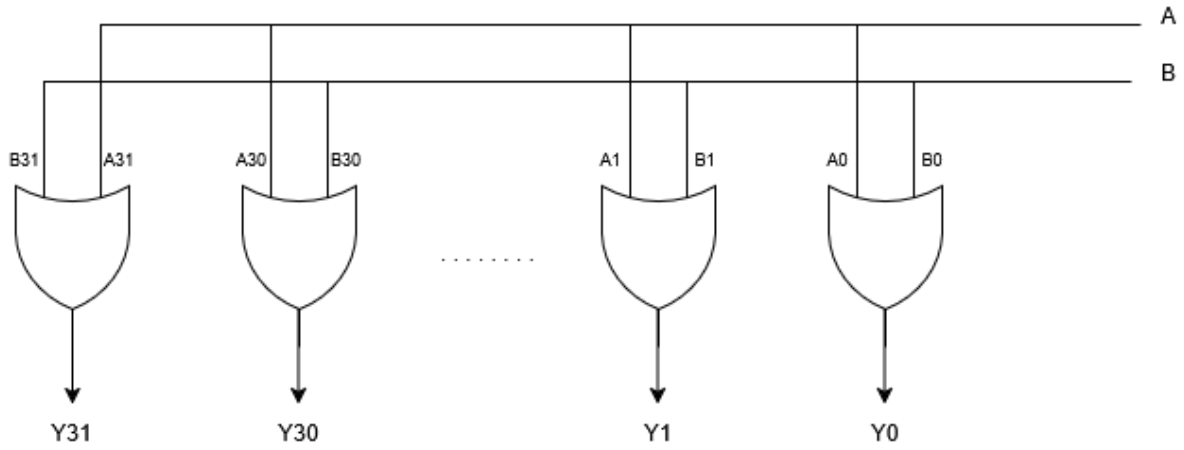

Figure 10. OR for 32 bits

4.2.3 NOT

To implement this operation I used 32 inverters, one for each bit of the operand.

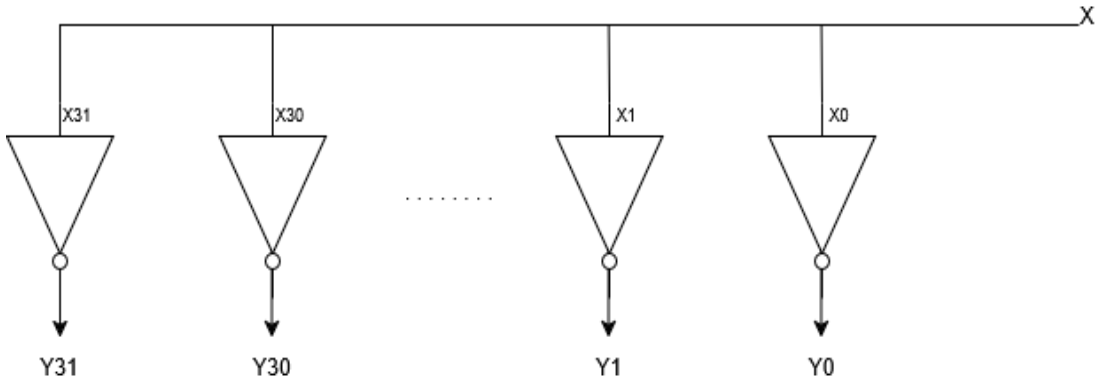

Figure 11. NOT for 32 bits

4.2.4 Rotate Left (ROL)

I will use 32 D-flip flops to store the operands and 32 multiplexers 2:1, one for each bit of the operand, to select the appropriate input bit after the rotation. Each multiplexer routes the bit from a higher-order position to its new location, with the most significant bits wrapping around to the least significant positions.

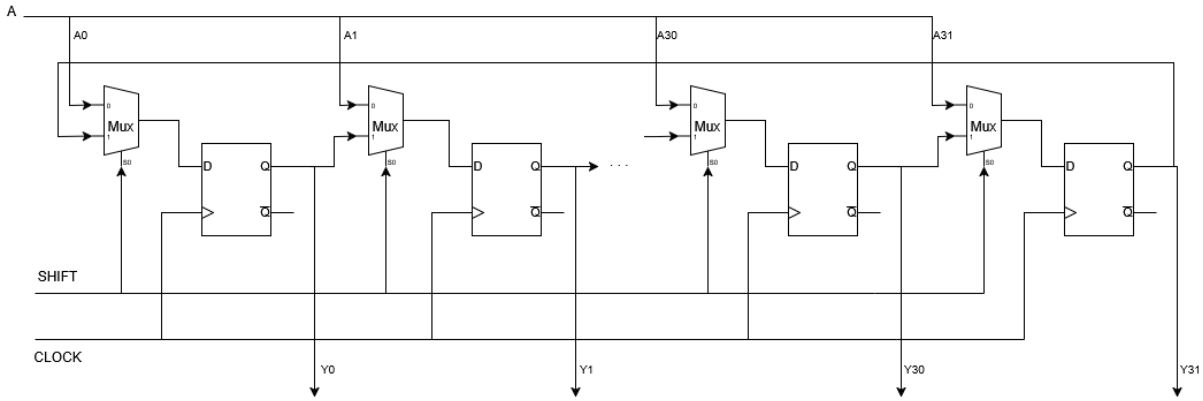

Figure 12. Shift left

4.2.5 Rotate Right (ROR)

For this Operation we use the same approach as with the Rotate Left operation but change the direction of the rotation. The multiplexers route the lower-order bits to the higher-order positions.

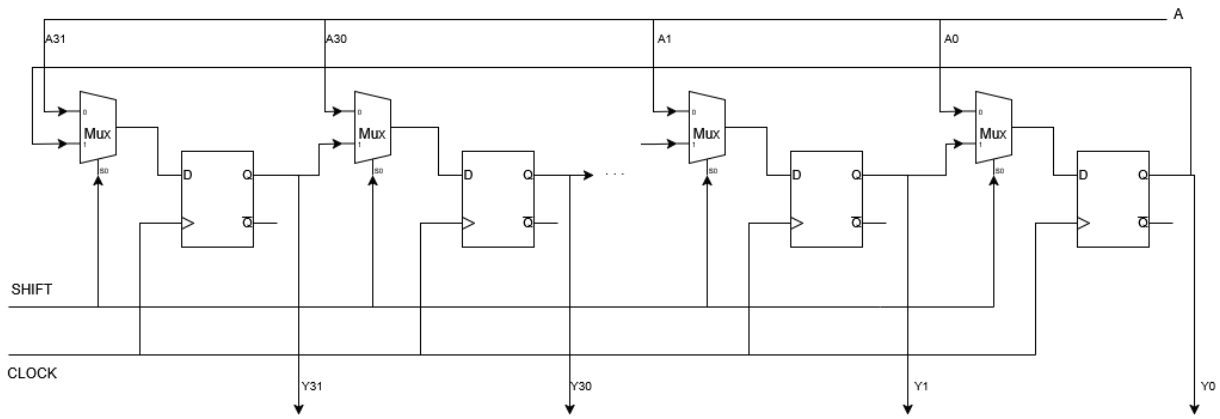

Figure 13. Shift right

5. Implementation

This chapter details the practical realization of the ALU described in the previous analysis section, utilizing VHDL within the Xilinx Vivado environment.

The implementation follows a modular hierarchical architecture, where the Arithmetic Unit, Logic Unit, Shifter, Multiplier, and Divider are developed as independent components interconnected within a top-level entity, the Execution Unit. The system operates on 32-bit Two's Complement integers and is managed by a central Control Unit, which handles instruction decoding and synchronizes data flow between the memory, accumulator, and execution blocks.

5.1 Arithmetic Unit

The Arithmetic Unit is designed as a combinational circuit that performs addition, subtraction, increment, decrement, and negation. Structurally, it consists of 32 one-bit full adders connected in series using a VHDL generate statement to form a 32-bit Ripple Carry Adder.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity arithmetic_unit is
    Port (
        X : in  STD_LOGIC_VECTOR (31 downto 0);
        Y : in  STD_LOGIC_VECTOR (31 downto 0);
        ctrl_inv_x : in  STD_LOGIC;
        ctrl_inv_y : in  STD_LOGIC;
        ctrl_use_y : in  STD_LOGIC;
        ctrl_cin : in  STD_LOGIC;
        flags: out STD_LOGIC_VECTOR (6 downto 0)
    );
end arithmetic_unit;
```

Figure 14. Arithmetic Unit component

Rather than using separate hardware for each operation, I chose to use a single adder controlled by four signals that manipulate the operands before calculation:

- ctrl_inv_x: is used to invert the input X during negation.
- ctrl_inv_y: is used to invert the second input Y for subtraction and decrement.
- ctrl_use_y: used to set Y to zero for the operations that don't need it (increment, negation) or need it to be zero (decrement).
- ctrl_cin: sets the carry (for subtraction, incrementation, negation).

5.2 Logic Unit

The Logic Unit is implemented as a purely combinational circuit responsible for executing the AND, OR, and NOT operations. It outputs the result of the performed operation and the flag vector.

```
entity logic_unit is
    Port (
        X : in  STD_LOGIC_VECTOR (31 downto 0);
        Y : in  STD_LOGIC_VECTOR (31 downto 0);
        operation : in  STD_LOGIC_VECTOR (1 downto 0);
        result : out STD_LOGIC_VECTOR (31 downto 0);
        flags: out STD_LOGIC_VECTOR (6 downto 0)
    );
end logic_unit;
```

Figure 15. Logic Unit component

The code employs a process block with a case statement to multiplex the operations based on the 2-bit operation control signal. This behavioral description models the array of 32 parallel logic gates described in the analysis and design chapters. For the NOT operation, the unit inverts the accumulator input (X), ignoring the second operand (Y).

5.3 Shifter

The Shifter is implemented as a sequential synchronous circuit, utilizing a 32-bit register composed of D-flip flops to store the current operand.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity shifter is
    Port (
        clk : in  STD_LOGIC;
        rst : in  STD_LOGIC;
        load : in  STD_LOGIC;
        enable : in  STD_LOGIC;
        shift_dir : in  STD_LOGIC;
        data_in : in  STD_LOGIC_VECTOR (31 downto 0);
        result : out STD_LOGIC_VECTOR (31 downto 0)
    );
end shifter;
```

Figure 15. Shifter component

The component handles the Shift Right and Shift Left operations, depending on the control signals:

- load: enable signal that allows the shifter to accept new data from data_in.
- enable: triggers the rotation operation.
- shift_dir: controls the rotation direction (0 for Shift Left, 1 for Shift Right).

5.4 Multiplier

The Multiplier is implemented as a sequential digital circuit that executes the Shift-and-Add algorithm described in the Analysis chapter. The module operates on two 32-bit input vectors, X (Multiplicand) and Y (Multiplier), and produces a 32-bit result.

```
entity multiplier is
    Port (
        clk : in STD_LOGIC;
        rst : in STD_LOGIC;
        start : in STD_LOGIC;
        X_in : in STD_LOGIC_VECTOR (31 downto 0);
        Y_in : in STD_LOGIC_VECTOR (31 downto 0);
        result : out STD_LOGIC_VECTOR (31 downto 0);
        flags : out STD_LOGIC_VECTOR (6 downto 0);
        ready : out STD_LOGIC
    );
end multiplier;
```

Figure 16. Multiplier component

The design utilizes a Finite State Machine (FSM) with four distinct states: IDLE, INIT, OPERATE, and FINISH.

Initialization (INIT): Because the standard Shift-and-Add algorithm operates on unsigned magnitudes, the circuit first converts the signed Two's Complement inputs into their absolute values. The final sign of the product is pre-calculated by performing an XOR operation on the most significant bits (sign bits) of the two operands.

Operation Loop (OPERATE): The core calculation utilizes three internal registers:

B Register: Stores the absolute value of the Multiplicand.

Q Register: Stores the absolute value of the Multiplier.

A Register: Acts as the Accumulator for the partial products.

Finalization (FINISH): Once the counter reaches zero, the circuit applies the pre-calculated sign to the result. If the result should be negative, the value in the Accumulator is negated (converted back to Two's Complement) before being sent to the output and asserting the ready signal.

5.5 Divider

The Divider is implemented as a sequential circuit executing the Restoring Division algorithm. The VHDL implementation defines a state machine with states IDLE, INIT, OPERATE, and FINISH. The logic avoids the complexity of a separate hardware subtractor by reusing the arithmetic logic within the process block.

```
entity divider is
    Port (
        clk : in STD_LOGIC;
        rst : in STD_LOGIC;
        start : in STD_LOGIC;
        X_dividend : in STD_LOGIC_VECTOR (31 downto 0);
        Y_divisor : in STD_LOGIC_VECTOR (31 downto 0);
        quotient : out STD_LOGIC_VECTOR (31 downto 0);
        remainder : out STD_LOGIC_VECTOR (31 downto 0);
        flags : out STD_LOGIC_VECTOR (6 downto 0);
        ready : out STD_LOGIC
    );
end divider;
```

Figure 17. Divider component

Initialization: The circuit checks if the divisor is zero to prevent undefined behavior, setting a divide_by_zero flag if necessary. The inputs are converted to positive magnitudes, and the sign of the quotient is determined by the XOR of the operand signs, while the remainder inherits the sign of the dividend.

Restoring Division Logic (OPERATE): The algorithm uses a combined 64-bit shift register approach using registers A (Partial Remainder) and Q (Dividend/Quotient).

Output Generation: After 32 iterations, the values in Q and A represent the absolute Quotient and Remainder, respectively. The FSM applies the correct sign to each output based on the initial sign checks before asserting the ready signal.

5.6 Control Unit

The Control Unit serves as the central command module for the ALU, coordinating all operations by decoding instruction signals and generating the necessary control vectors for the execution units.

```
entity control_unit is
    Port (
        clk         : in  STD_LOGIC;
        rst         : in  STD_LOGIC;
        opcode      : in  STD_LOGIC_VECTOR (3 downto 0);
        start_op    : in  STD_LOGIC;

        -- Arithmetic Unit Signals
        ctrl_inv_x  : out STD_LOGIC;
        ctrl_inv_y  : out STD_LOGIC;
        ctrl_use_y  : out STD_LOGIC;
        ctrl_cin    : out STD_LOGIC;

        -- Logic Unit Signals
        logic_op    : out STD_LOGIC_VECTOR (1 downto 0);

        -- Shifter Signals
        shift_dir   : out STD_LOGIC;
        shift_en    : out STD_LOGIC;

        -- Multiplier/Divider Signals
        start_mul   : out STD_LOGIC;
        start_div   : out STD_LOGIC;

        -- Result Mux Select (To choose final output)
        -- 00: Arithmetic, 01: Logic, 10: Shift, 11: Mul/Div
        mux_sel     : out STD_LOGIC_VECTOR (1 downto 0)
    );
end control_unit;
```

Figure 18. Control Unit

For each operation, the Control Unit asserts a unique combination of signals:

Arithmetic Controls: It generates ctrl_inv_x, ctrl_inv_y, ctrl_use_y, and ctrl_cin to configure the unified Adder/Subtractor block for the specific arithmetic task.

Logic Select: It outputs a 2-bit logic_op vector to select between AND, OR, and NOT gates within the Logic Unit.

Enable Signals: For sequential operations (Multiplication, Division, and Shifting), it routes the start signal to the specific enable port (start_mul, start_div, shift_en) of the required component.

Result Multiplexing: To ensure the correct result is written to the Accumulator, the Control Unit generates a 2-bit mux_sel signal. This signal controls the output multiplexer.

5.7 Execution Unit

The Execution Unit serves as the top-level controller for the 32-bit Arithmetic Logic Unit. It integrates and manages the five functional blocks: the Arithmetic Unit, Logic Unit, Shifter, Multiplier, and Divider and the Control Unit.

```
entity execution_unit is
    Port (
        clk, rst    : in  STD_LOGIC;
        opcode      : in  STD_LOGIC_VECTOR (3 downto 0);
        start_op    : in  STD_LOGIC;
        acc_data_in : in  STD_LOGIC_VECTOR (31 downto 0);
        load_acc    : in  STD_LOGIC;
        op2_data_in : in  STD_LOGIC_VECTOR (31 downto 0);
        load_op2    : in  STD_LOGIC;
        alu_result  : out STD_LOGIC_VECTOR (31 downto 0);
        alu_flags   : out STD_LOGIC_VECTOR (6 downto 0);
        busy        : out STD_LOGIC
    );
end execution_unit;
```

Figure 19. Execution Unit

The design follows an accumulator-based architecture, where a 32-bit register (ACC) stores the result of operations and acts as the primary operand for subsequent instructions. The unit features a timing controller that supports both single-cycle execution (ADD, SUB, AND, OR) and multi-cycle execution (MUL, DIV), utilizing a busy/ready handshake protocol to ensure data integrity.

5.8 Memory Unit

The Instruction Memory component is a Read-Only Memory (ROM) module designed to help with hardware verification on the FPGA board (Basys 3). It allows the selection of 32-bit test cases using a 6-bit address selector.

```
entity instruction_memory is
    Port (
        addr        : in  STD_LOGIC_VECTOR(5 downto 0);
        out_opcode  : out STD_LOGIC_VECTOR(3 downto 0);
        out_acc     : out STD_LOGIC_VECTOR(31 downto 0);
        out_op2     : out STD_LOGIC_VECTOR(31 downto 0)
    );
end instruction_memory;
```

Figure 20. Memory component

There are 35 test cases with 68 bits per entry, structured as follows:

- Operation Code (opcode, 4 bits)
- Accumulator Input (out_acc, 32 bits)
- Operand 2 Input (out_op2, 32 bits)

```
constant ROM : rom_type := (
    "0000" & x"0000000A" & x"00000010",-- 0: ADD 10 + 16
    "0000" & x"00000006" & x"0000000A",-- 1: ADD 6 + 10
    "0000" & x"FFFFFFFA" & x"FFFFFFF0",-- 2: ADD -6 + (-16)
    "0000" & x"7FFFFFFF" & x"6FFFFFFF",-- 3: ADD Overflow Test
    "0001" & x"0000000A" & x"00000010",-- 4: SUB 10 - 16
    "0001" & x"0000000A" & x"00000012",-- 5: SUB 10 - 18
    "0001" & x"80000000" & x"00000002",-- 6: SUB Underflow Test
    "0010" & x"0000000A" & x"00000000",-- 7: INC 10 + 1
    "0010" & x"80000002" & x"00000000",-- 8: INC Negative + 1
    "0011" & x"FFFFFFF8" & x"00000000",-- 9: DEC -8 - 1
    "0011" & x"80000000" & x"00000000",-- 10: DEC Min Int - 1
    "0100" & x"0000000A" & x"00000000",-- 11: NEG 10
    "0100" & x"FFFFFFF6" & x"00000000",-- 12: NEG -10
    "0101" & x"0000000A" & x"00000009",-- 13: AND 10 & 9
    "0101" & x"ABC500F1" & x"0A536221",-- 14: AND Large Hex
    "0110" & x"0000000A" & x"0000000B",-- 15: OR 10 | 11
    "0110" & x"ABC500F1" & x"0A536221",-- 16: OR Large Hex
    "0111" & x"0000000A" & x"00000000",-- 17: NOT 10
    "0111" & x"FFFFFFF5" & x"00000000",-- 18: NOT -11
    "1000" & x"80000000" & x"00000000",-- 19: ROL MSB to LSB
    "1000" & x"0000000B" & x"00000000",-- 20: ROL 11 << 1
    "1001" & x"00000010" & x"00000000",-- 21: ROR 16 >> 1
    "1001" & x"0000000B" & x"00000000",-- 22: ROR 11 >> 1
    "1010" & x"0000000A" & x"00000006",-- 23: MUL 10 * 6
    "1010" & x"FFFFFFF6" & x"00000006",-- 24: MUL -10 * 6
    "1010" & x"0000000A" & x"FFFFFFFA",-- 25: MUL 10 * -6
    "1010" & x"FFFFFFF6" & x"FFFFFFFA",-- 26: MUL -10 * -6
    "1010" & x"0000000A" & x"00000000",-- 27: MUL Zero
    "1010" & x"7FFFFFFF" & x"00000003",-- 28: MUL Overflow
    "1011" & x"0000000C" & x"00000006",-- 29: DIV 12 / 6
    "1011" & x"0000000C" & x"00000005",-- 30: DIV 12 / 5
    "1011" & x"FFFFFFF4" & x"00000005",-- 31: DIV -12 / 5
    "1011" & x"0000000C" & x"FFFFFFFB",-- 32: DIV 12 / -5
    "1011" & x"FFFFFFF4" & x"FFFFFFFB",-- 33: DIV -12 / -5
    "1011" & x"0000000C" & x"00000000",-- 34: DIV Divide by Zero
    others => (others => '0')
);
```

Figure 21. Memory addresses

5.9 Debouncer

The Debouncer is a necessary component for reliable interaction with the physical buttons of the FPGA board. It acts as a digital filter using a counter to verify that the input signal remains stable for a defined period before changing the output state.

```
entity debouncer is
    Port ( clk : in STD_LOGIC;
           btn_in : in STD_LOGIC;
           btn_out : out STD_LOGIC);
end debouncer;
```

Figure 22. Debouncer

If the input changes, a counter resets. The output (btn_out) is updated only if the input remains stable until the counter reaches its threshold. The result is a clean, single pulse signal suitable for the Execution Unit's control inputs.

5.10 Seven Segment Display Driver

The Seven Segment Driver is responsible for visualizing the results on the Basys 3 development board. Since the board only has four 7-segment digits, it can't display a full 32-bit integer.

```
entity seven_seg_driver is
    Port ( clk : in STD_LOGIC;
           rst : in STD_LOGIC;
           display_data : in STD_LOGIC_VECTOR (15 downto 0);
           an : out STD_LOGIC_VECTOR (3 downto 0);
           seg : out STD_LOGIC_VECTOR (6 downto 0)
           );
end seven_seg_driver;
```

Figure 23. SSD Driver

5.11 Top Level Module

The Top Level Unit serves as the hardware wrapper for the project, bridging the internal logic with the FPGA. It instantiates the Execution Unit, Instruction Memory, Debouncers, and the Display Driver.

```
entity top_level is
    Port (
        clk : in STD_LOGIC;
        sw  : in STD_LOGIC_VECTOR (5 downto 0);
        btnC : in STD_LOGIC;
        btnU : in STD_LOGIC;
        btnR : in STD_LOGIC;
        btnD : in STD_LOGIC;

        seg : out STD_LOGIC_VECTOR (6 downto 0);
        an  : out STD_LOGIC_VECTOR (3 downto 0);
        led : out STD_LOGIC_VECTOR (15 downto 0)
    );
end top_level;
```

Figure 24. Top-Level of the ALU

Functionalities implemented in the Top Level include:

- **Instruction Selection:** Maps the switches of the board to the address bus of the Instruction Memory, allowing the user to select specific test cases.
- **Manual Control:** Connects the debounced buttons to the Execution Unit's control signals:
    - **Load (btnD):** Manually loads the accumulator or operand registers with data from memory.
    - **Execute (btnC):** Triggers the start_op signal to begin the ALU operation.
    - **Reset (btnU):** Resets all components.
- **Display Logic:** Because the physical display is limited to 16 bits, the Top Level implements a toggling mechanism. By pressing a button (btnR), the user can switch the view between the Lower 16 bits and the Upper 16 bits of the 32-bit result.
- **Status Indication:** Maps the ALU flags and the current Opcode to the on-board LEDs.

6. Testing and Validation

This chapter documents the verification process for the 32-bit ALU. The objective is to validate the correct functionality of all arithmetic, logical, and sequential operations. The verification process ensures that the design not only performs standard calculations correctly but also robustly handles edge cases, such as overflow and division by zero.

Operatio	Operand 1 (Dec	Operand 1 (Hex	Operand 2 (Dec	Operand 2 (Hex	Result (Dec	Result (Hex
ADD (+)	10	A	16	10	26	1A
ADD (+)	6	6	10	A	16	10
ADD (+)	-6	FFFFFFFA	-16	FFFFFFF0	-22	FFFFFFEA
ADD (+)	2147483647	7FFFFFFF	1879048191	6FFFFFFF	-268435458	EFFFFFFE
SUB (-)	10	A	16	10	-6	FFFFFFFA
SUB (-)	10	A	18	12	-8	FFFFFFF8
SUB (-)	-2147483648	80000000	2	2	2147483646	7FFFFFFE
INC (+1)	10	A	-	-	11	B
INC (+1)	-2147483646	80000002	-	-	-2147483645	80000003
DEC (-1)	-8	FFFFFFF8	-	-	-9	FFFFFFF7
DEC (-1)	-2147483648	80000000	-	-	2147483647	7FFFFFFF
NEG (-x)	10	A	-	-	-10	FFFFFFF6
NEG (-x)	-10	FFFFFFF6	-	-	10	A
AND	10	A	9	9	8	8
AND	2881814769	ABC500F1	173236769	0A536221	172032033	0A410021
OR	10	A	9	9	11	B
OR	2881814769	ABC500F1	173236769	0A536221	2883019505	ABD762F1
NOT	10	A	-	-	-11	FFFFFFF5
NOT	-11	FFFFFFF5	-	-	10	A
ROL	2147483648	80000000	-	-	1	1
ROL	11	B	-	-	22	16
ROR	16	10	-	-	8	8
ROR	11	B	-	-	2147483653	80000005
MUL (×)	10	A	6	6	60	3C
MUL (×)	-10	FFFFFFF6	6	6	-60	FFFFFFC4
MUL (×)	10	A	-6	FFFFFFFA	-60	FFFFFFC4
MUL (×)	-10	FFFFFFF6	-6	FFFFFFFA	60	3C
MUL (×)	10	A	0	0	0	0
MUL (×)	2147483647	7FFFFFFF	3	3	2147483645	7FFFFFFD
DIV (/)	12	C	6	6	2	2
DIV (/)	12	C	5	5	2	2
DIV (/)	-12	FFFFFFF4	5	5	-2	FFFFFFFE
DIV (/)	12	C	-5	FFFFFFFB	-2	FFFFFFFE
DIV (/)	-12	FFFFFFF4	-5	FFFFFFFB	2	2
DIV (/)	12	C	0	0	0	0

Figure 25. Table of test cases

The validation was performed using the Vivado Simulator. A VHDL testbench (tb_execution_unit.vhd) was developed to simulate the operations of the ALU, as described in Figure 25. The results of the simulation can be seen below:

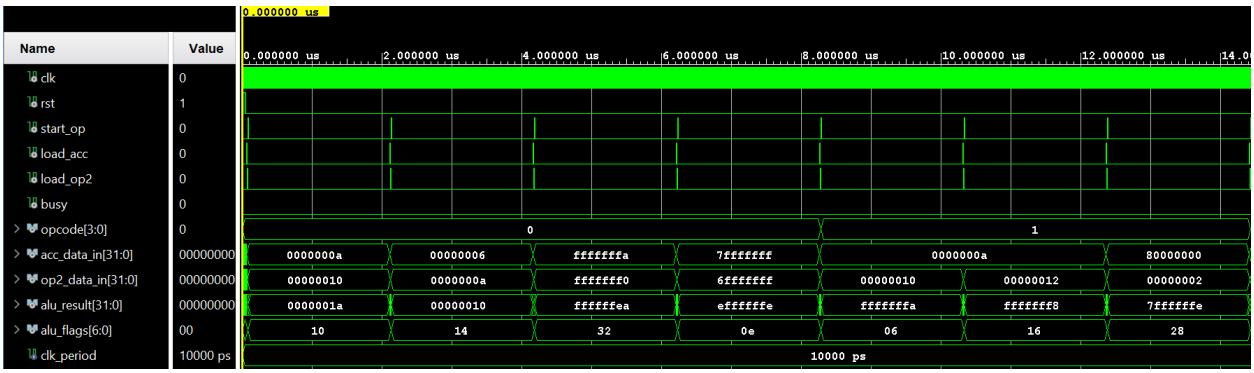

Figure 26.1. Simulation results

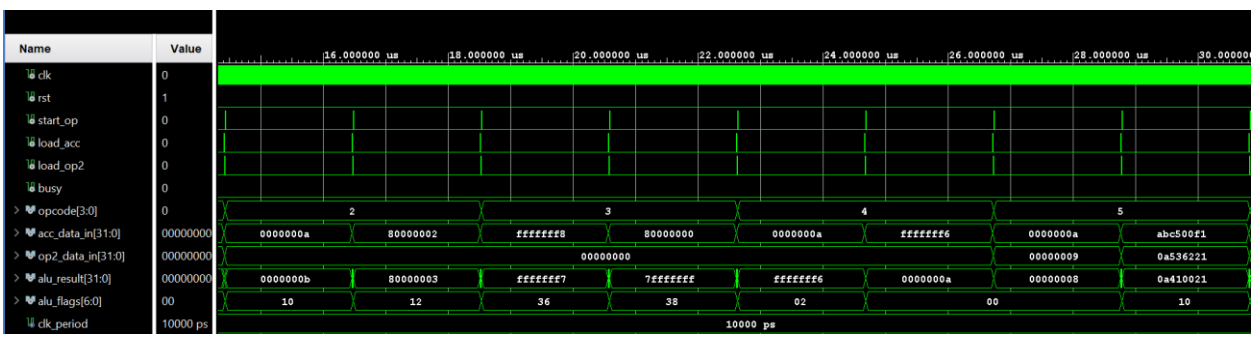

Figure 26.2. Simulation results

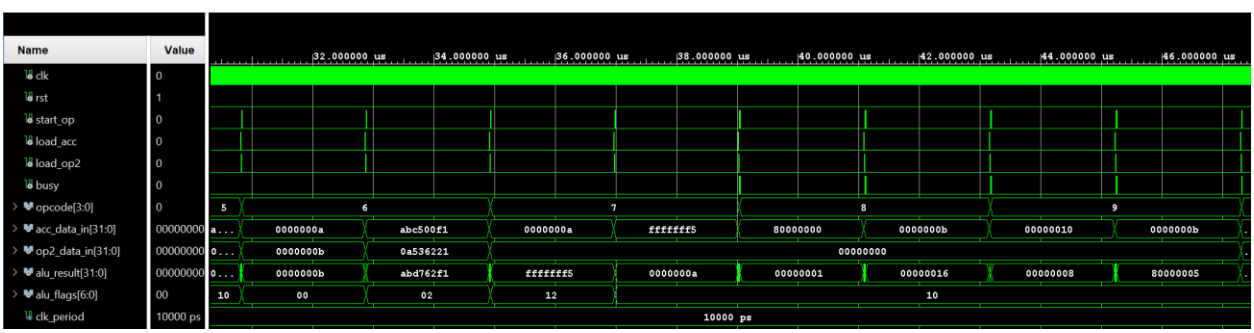

Figure 26.3. Simulation results

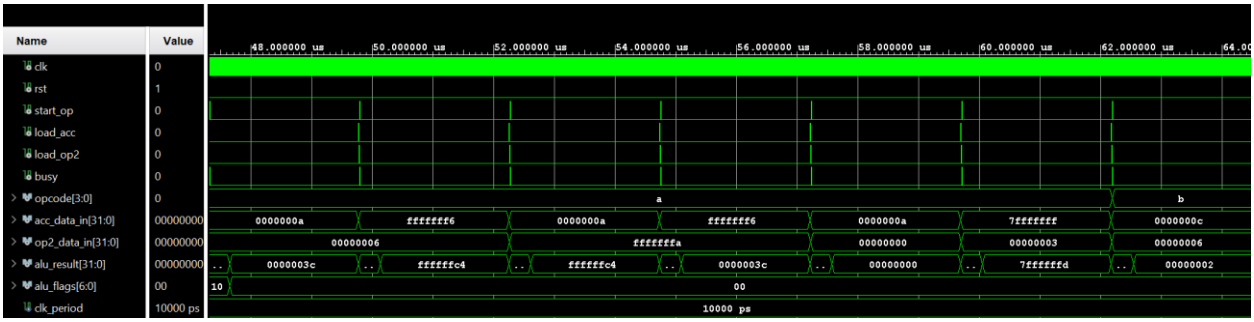

Figure 26.4. Simulation results

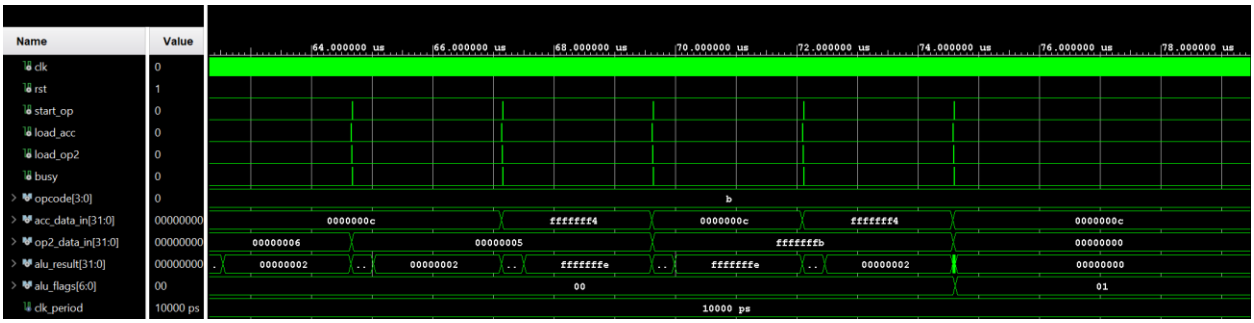

Figure 26.5. Simulation results

7. Conclusion

The primary objective of this project was to design, implement, and validate a 32-bit Arithmetic Logic Unit (ALU) using VHDL and Vivado. The final design successfully integrated five distinct functional blocks: Arithmetic, Logic, Shifter, Multiplier, and Divider into an Execution Unit capable of performing operations on signed Two's Complement integers.

The implementation allowed independent development and testing of each component. The design efficiently reused hardware resources: the Arithmetic Unit utilized a single 32-bit adder to perform addition, subtraction, increment, decrement, and negation through the manipulation of control signals. Complex sequential operations like Multiplication and Division were implemented using algorithmic state machines (FSMs) to handle the multi-cycle shift-and-add and restoring division processes.

In conclusion, the project resulted in a fully validated hardware ALU. The integration of a memory-based test vector generator and a multiplexed 7-segment display driver

allowed for real-time, interactive verification of the design, demonstrating the practical application of digital logic principles.

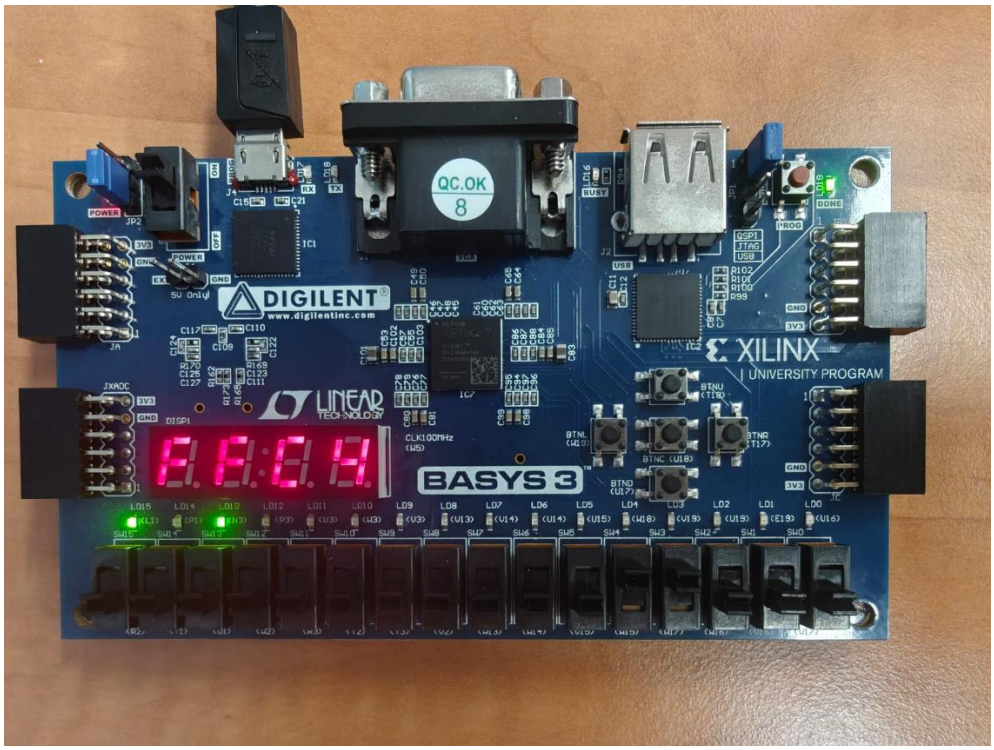

Figure 27. The FPGA board

8. References

[1] *Arithmetic logic unit*, https://en.wikipedia.org/wiki/Arithmetic_logic_unit

[2] *What Is an Arithmetic Logic Unit (ALU)? Meaning, Components, and Functions,* https://www.spiceworks.com/tech/hardware/articles/what-is-alu/#h_1867108352168234359983 5

[3] *What is an arithmetic logic unit (ALU) and how does it work?,* https://www.techtarget.com/whatis/definition/arithmetic-logic-unit-ALU

[4] *First Draft of a Report on the EDVAC,* https://en.wikipedia.org/wiki/First_Draft_of_a_Report_on_the_EDVAC

[5] *Whirlwind I,* https://en.wikipedia.org/wiki/Whirlwind_I

[6] *AMD Am2900,* https://en.wikipedia.org/wiki/AMD_Am2900

[7] *74181,* https://en.wikipedia.org/wiki/74181

[8] *4.2 ALU design and implementation,* https://fiveable.me/introduction-computer-architecture/unit-4/alu-design-implementation/study-guide/M5RQtjUjXhSaOhgd

[9] *The Z-80 has a 4-bit ALU. Here's how it works,* https://www.righto.com/2013/09/the-z-80-has-4-bit-alu-heres-how-it.html

[10] Abdul Rehman Buzdar, Liguo Sun, Abdullah Buzdar, *Comparative Analysis of ALU Implementation with RCA and Sklansky Adders in ASIC Design Flow, International Journal of Advanced Computer Science and Applications, Vol. 7, No. 7, 2016, https://thesai.org/Downloads/Volume7No7/Paper_67-Comparative_Analysis_of_ALU_Implementation.pdf*

[11] *Representations of Logic Operations*, https://www.realdigital.org/doc/e127ebfa82dbc904b5c0dac5d1adce8e

[12] *Logic Gates - Building an ALU,* http://www.csc.villanova.edu/~mdamian/Past/csc2400fa13/assign/ALU.html

[13] *Two's complement, https://en.wikipedia.org/wiki/Two%27s_complement*